

ING. SISTEMAS 2025-1

Programación Orientada a Objetos

Trabajo investigativo para Estructura de Datos

Abraham Ospina Villa
ID: 966825

Tabla de Contenidos

1. Introducción
2. Objetivos
3. Programación Orientada a Objetos
 - Conceptos fundamentales
 - Principios de la POO
 - Beneficios y aplicaciones
4. Conclusiones
5. Referencias Bibliográficas

Introducción

La Programación Orientada a Objetos (POO) es un paradigma de programación que se basa en el uso de "objetos", los cuales representan entidades del mundo real. Esta metodología permite mejorar la organización y reutilización del código, facilitando el desarrollo y mantenimiento de software.

Objetivos

- Comprender los principios fundamentales de la Programación Orientada a Objetos.
- Identificar los beneficios de la POO en el desarrollo de software.
- Analizar ejemplos prácticos de implementación de la POO.

Conceptos fundamentales

- Clases: Son instancias de una clase. Cada objeto tiene su propio estado, representado por los valores de sus atributos, y puede ejecutar acciones definidas en su clase.

- Objetos: Son instancias de una clase. Cada objeto tiene su propio estado, representado por los valores de sus atributos, y puede ejecutar acciones definidas en su clase.

- Atributos: Son las propiedades o características que describen un objeto. Cada objeto puede tener valores diferentes para sus atributos, lo que lo hace único.

- Métodos: Son las acciones o comportamientos que un objeto puede realizar. Se definen dentro de la clase y pueden manipular los atributos del objeto o interactuar con otros objetos.

Principios de la POO

1. Encapsulamiento: Este principio permite restringir el acceso a ciertos detalles de un objeto y exponer solo lo necesario. Se logra a través de modificadores de acceso como "privado" o "público" en lenguajes como Java y C++.
2. Herencia: Permite que una clase "hija" adquiera los atributos y métodos de una clase "padre". Esto fomenta la reutilización del código y facilita la extensibilidad del sistema.
3. Polimorfismo: Hace posible que diferentes objetos respondan de manera distinta al mismo método. Existen dos tipos de polimorfismo: el polimorfismo en tiempo de compilación (sobrecarga de métodos) y el polimorfismo en tiempo de ejecución (sobreescripción de métodos).
4. Abstracción: Permite modelar objetos del mundo real destacando solo sus características esenciales y ocultando los detalles irrelevantes. Se implementa a través de clases y métodos abstractos.

Beneficios y aplicaciones

- Modularidad: Permite dividir un programa en componentes más pequeños y manejables.
- Reutilización del código: Gracias a la herencia, se pueden reutilizar clases existentes en nuevos programas.
- Mantenimiento eficiente: La estructura modular facilita la actualización y corrección de errores.
- Mayor seguridad: El encapsulamiento protege los datos y evita modificaciones no deseadas.

Conclusiones

La Programación Orientada a Objetos es un paradigma esencial en el desarrollo de software moderno. Sus principios permiten crear sistemas más modulares, reutilizables y fáciles de mantener, mejorando la eficiencia y calidad del código

Referencias Bibliográficas

- Deitel, P., & Deitel, H. (2016). *Java: Como Programar*. Pearson Educación.
- Sierra, K., & Bates, B. (2005). *Head First Java*. O'Reilly Media.
- Prieto, J. (2010). *Programación Orientada a Objetos en Java*. Alfaomega Grupo Editor.
- Pérez, R. (2014). *Fundamentos de Programación Orientada a Objetos con Python*. Marcombo.
- Joyanes, L. (2011). *Programación en C++ y Java: Fundamentos del Desarrollo de Software Orientado a Objetos*. McGraw-Hill.